DESENVOLVIMENTO FULL STACK

ALEX ZACHARIAS

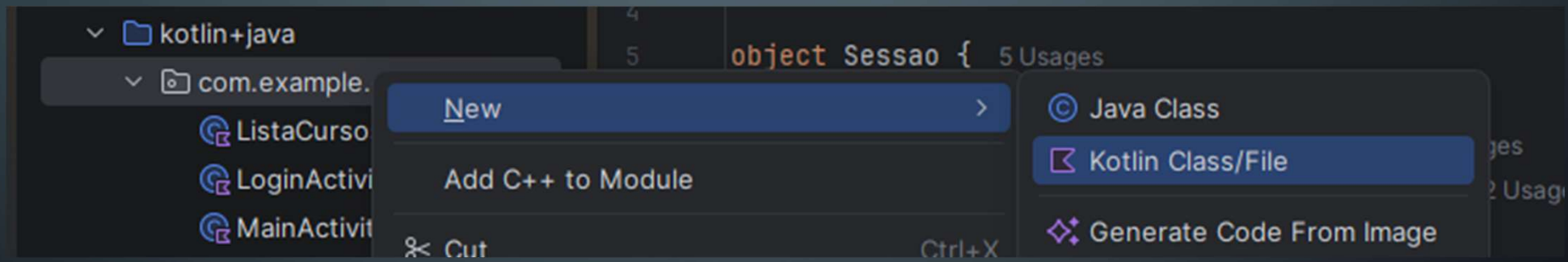
ALEX@EINSTEINLIMEIRA.COM.BR

ACESSANDO NOSSA API JWT

- Vamos acessar nossa API com JWT.
- Já temos um APP acessando a API simples (sem JWT), vamos altera-lo para incluir a validação JWT.
- Primeiro passo é criar um object (classe estática) para armazenar o token e usuário.
- Vamos utilizar a classe Context do Android para acessar recursos do ambiente e guardar informações.

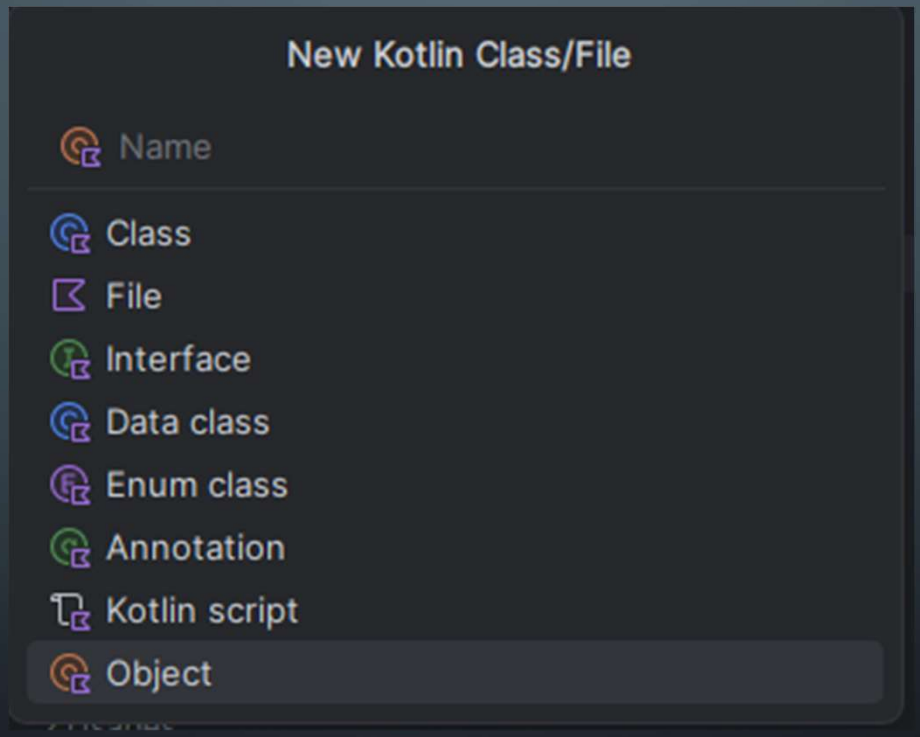
ACESSANDO NOSSA API JWT

- Com o botão direito, clique sobre a pasta onde estão os arquivos fontes e vá em new – Kotlin Class/File:



ACESSANDO NOSSA API JWT

- Selecione Object e chame o mesmo de Sessao



ACESSANDO NOSSA API JWT

- No arquivo criado, inclua o código abaixo:

```
package com.example.escola2

import android.content.Context

object Sessao {  5 Usages
    private const val PREFS = "sessao"  1 Usage
    private const val KEY_TOKEN = "token"  2 Usages
    private const val KEY_USUARIO = "usuario"  2 Usages

    private fun prefs(context: Context) =  4 Usages
        context.getSharedPreferences( p0 = PREFS,  p1 = Context.MODE_PRIVATE)

    fun salvar(context: Context, token: String, usuario: String) {  1 Usage
        prefs(context).edit()
            .putString( p0 = KEY_TOKEN,  p1 = token)
            .putString( p0 = KEY_USUARIO,  p1 = usuario)
            .apply()
    }

    fun token(context: Context): String? =  2 Usages
        prefs(context).getString( p0 = KEY_TOKEN,  p1 = null)

    fun usuario(context: Context): String? =
        prefs(context).getString( p0 = KEY_USUARIO,  p1 = null)

    fun autenticado(context: Context): Boolean =  2 Usages
        !token(context).isNullOrBlank()

    fun limpar(context: Context) {  1 Usage
        prefs(context).edit().clear().apply()
    }
}
```

ACESSANDO NOSSA API JWT

- Explicando o código:

```
3    import android.content.Context
```

- Importa a classe Context que permite acessar recursos e serviços do Android como armazenamento de preferencias.

```
5    object Sessao {
```

- Cria um object, uma classe estática que não precisa ser instanciada para ser utilizada.

ACESSANDO NOSSA API JWT

- Explicando o código:

```
3    import android.content.Context
```

- Importa a classe Context que permite acessar recursos e serviços do Android como armazenamento de preferencias.

```
5    object Sessao {
```

- Cria um object, uma classe estática que não precisa ser instanciada para ser utilizada.

ACESSANDO NOSSA API JWT

```
6       private const val PREFS = "sessao"  1 Usage
7       private const val KEY_TOKEN = "token"  2 Usages
8       private const val KEY_USUARIO = "usuario"  2 Usages
```

- Cria as constantes, atributos do object para receber os valores que iremos guardar: a sessão, o usuário e o token.

```
10      private fun prefs(context: Context) =  4 Usages
11          context.getSharedPreferences( p0 = PREFS,  p1 = Context.MODE_PRIVATE)
```

- Método que acessa o armazenamento local do Android chamado SharedPreferences, ele permite guardar pequenas informações no dispositivo em formato de chave e valor.
- Context.MODE_PRIVATE – determina que esses dados armazenados são privados para o aplicativo que o criou.

ACESSANDO NOSSA API JWT

```
13      fun salvar(context: Context, token: String, usuario: String) {
14          prefs(context).edit()
15              .putString( p0 = KEY_TOKEN,  p1 = token)
16              .putString( p0 = KEY_USUARIO,  p1 = usuario)
17              .apply()
18      }
```

- Salva os dados no dispositivo, recebe o contexto, o token e o usuário.
- .edit() inicia a edição das preferencias no contexto selecionado
- .putString(chave,valor) armazena o valor recebido na chave informada
- .apply() aplica as alterações no armazenamento de forma assíncrona, ou seja, sem bloquear a thread principal.

ACESSANDO NOSSA API JWT

```
20      fun token(context: Context): String? =  2 Usages
21          prefs(context).getString( p0 = KEY_TOKEN,  p1 = null)
```

- Retorna o token armazenado.
- String? A interrogação faz com que ele aceite retorna nulo caso ainda não tenha valor armazenado para o token.
- .getString(chave,null) retorna o valor armazenado na chave, caso a mesma não tenha valor, no lugar do null pode ser colocado um valor padrão para retorno.

ACESSANDO NOSSA API JWT

```
23      fun usuario(context: Context): String? =
24          prefs(context).getString( p0 = KEY_USUARIO,  p1 = null)
25
26      fun autenticado(context: Context): Boolean =  2 Usages
27          !token(context).isNullOrBlank()
```

- Mesma ideia da função anterior, mas a primeira retorna o usuário e a segunda retornar verdadeiro ou falso indicando que o usuário esta autenticado.
- Essa função autenticado() não verifica se o token expirou.

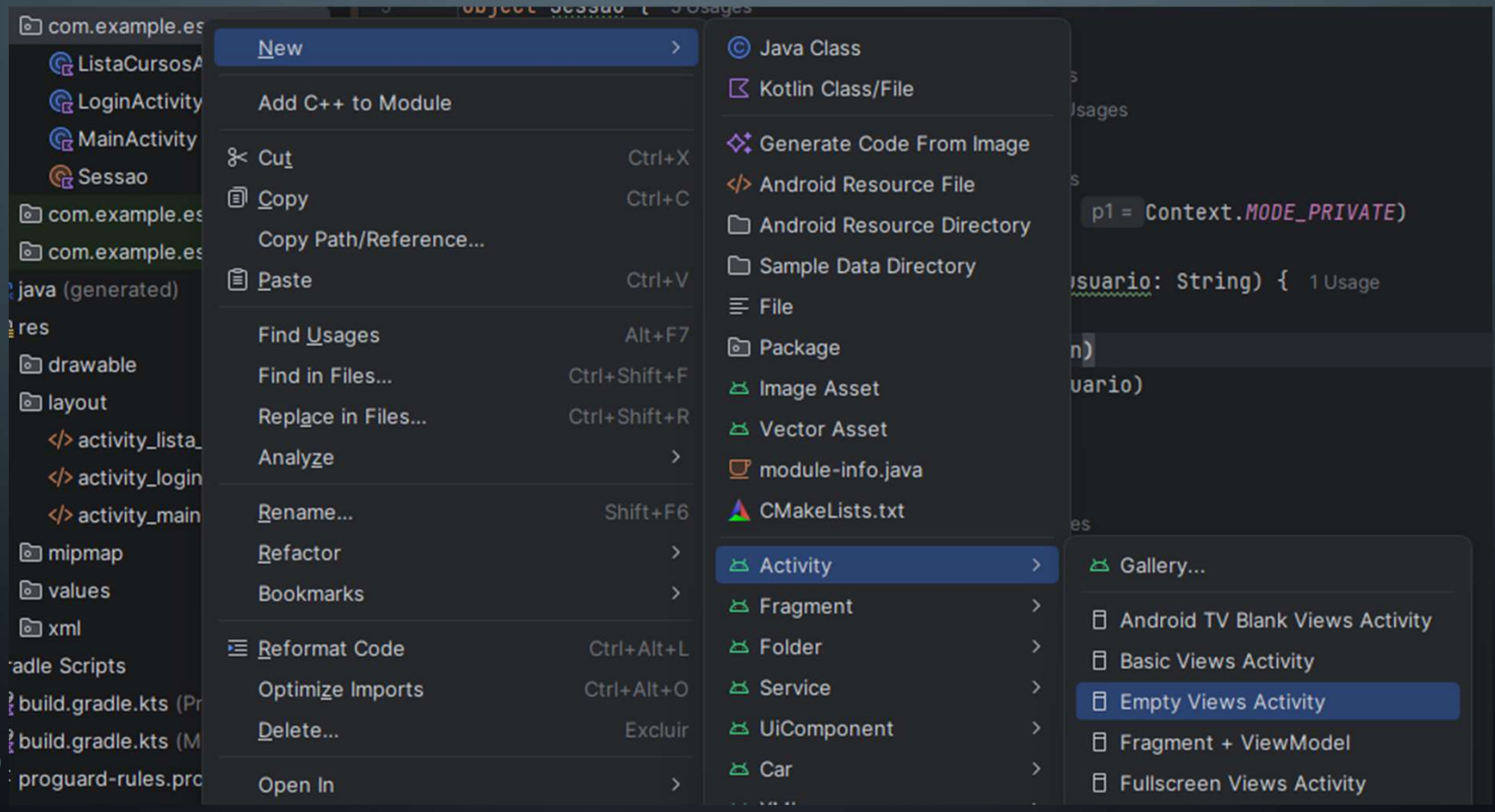
ACESSANDO NOSSA API JWT

```
29      fun limpar(context: Context) {  1 Usage
30          prefs(context).edit().clear().apply()
31      }
```

- Apaga os dados da sessão armazenados.

ACESSANDO NOSSA API JWT

- Agora vamos incluir a tela de login:



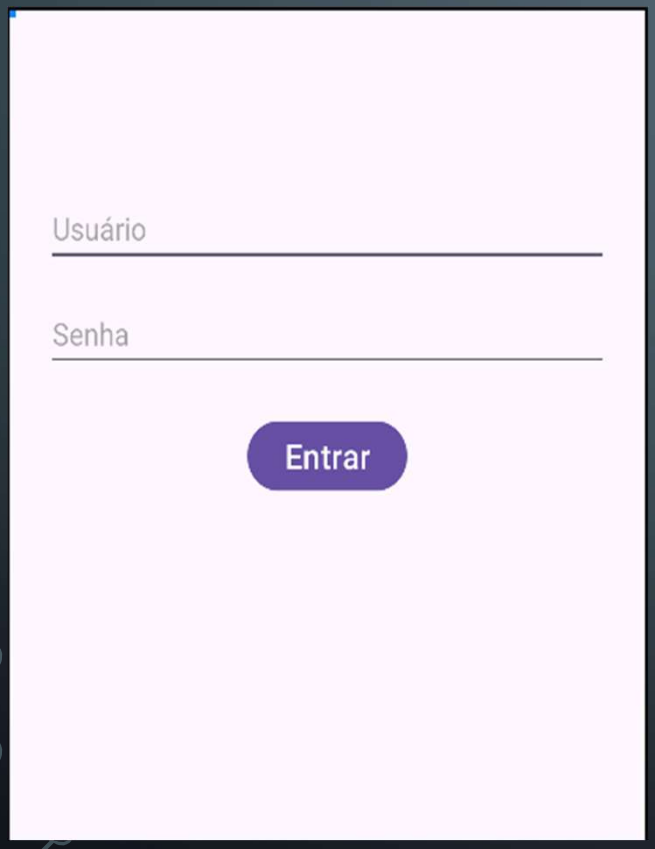
ACESSANDO NOSSA API JWT

- Vamos chama-la de LoginActivity:



ACESSANDO NOSSA API JWT

- Vamos configurar o xml da nossa tela de login (Activity_login)




```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center_horizontal"
    android:padding="24dp"
    tools:context=".LoginActivity">

    <EditText
        android:id="@+id/edit_login"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="80dp"
        android:hint="Usuário"
        android:inputType="text" />
```

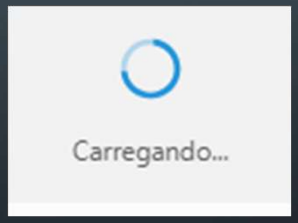
ACESSANDO NOSSA API JWT

```
20
21      <EditText
22          android:id="@+id/edit_senha"
23          android:layout_width="match_parent"
24          android:layout_height="wrap_content"
25          android:layout_marginTop="16dp"
26          android:hint="Senha"
27          android:inputType="textPassword" />
28
29      <Button
30          android:id="@+id/button_entrar"
31          android:layout_width="wrap_content"
32          android:layout_height="wrap_content"
33          android:layout_gravity="center_horizontal"
34          android:layout_marginTop="24dp"
35          android:text="Entrar"
36          android:textSize="20sp" />
```

```
37
38      <ProgressBar
39          android:id="@+id/progress_bar"
40          android:layout_width="wrap_content"
41          android:layout_height="wrap_content"
42          android:layout_marginTop="16dp"
43          android:visibility="gone" />
44
45  </LinearLayout>
46
```

ACESSANDO NOSSA API JWT

- Estamos utilizando um componente que nunca havíamos utilizado, o ProgressBar.
- Ele mostra uma barra circular para representar que o programa esta “carregando”.
- A propriedade visibility definida como gone faz com que o mesmo não apareça na tela ate ser alterado para visible.



ACESSANDO NOSSA API JWT

- Código da LoginActivity:

```
1   package com.example.escola2
2
3   import android.content.Intent
4   import android.os.Bundle
5   import android.view.View
6   import android.widget.Toast
7   import androidx.activity.enableEdgeToEdge
8   import androidx.appcompat.app.AppCompatActivity
9   import androidx.core.view.ViewCompat
10  import androidx.core.view.WindowInsetsCompat
11  import com.example.escola2.databinding.ActivityLoginBinding
12  import okhttp3.FormBody
13  import okhttp3.OkHttpClient
14  import okhttp3.Request
15  import org.json.JSONObject
16
17  class LoginActivity : AppCompatActivity(), View.OnClickListener {  4 Usages
18      private lateinit var binding: ActivityLoginBinding  9 Usages
19
```

ACESSANDO NOSSA API JWT

```
20    override fun onCreate(savedInstanceState: Bundle?) {
21        super.onCreate(savedInstanceState)
22        enableEdgeToEdge()
23        binding = ActivityLoginBinding.inflate(layoutInflater)
24        setContentView(binding.root)
25        ViewCompat.setOnApplyWindowInsetsListener( view = findViewById(R.id.main)) { v, insets ->
26            val systemBars = insets.getInsets( typeMask = WindowInsetsCompat.Type.systemBars())
27            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
28            insets
29        }
30        binding.buttonEntrar.setOnClickListener(this)
31
32        // Se já existe token salvo, pula o login
33        if (Sessao.autenticado( context = this))
34            abrirPrincipal()
35    }
36
```

- Só marquei as partes que são alteradas no código que já existia, nos próximos slides todo o código precisa ser incluído.

ACESSANDO NOSSA API JWT

```
37    override fun onClick(view: View) {
38        if (view.id == R.id.button_entrar)
39            btEntrar()
40    }
41
42    private fun btEntrar() {  1 Usage
43        val login = binding.editLogin.text.toString().trim()
44        val senha = binding.editSenha.text.toString()
45
46        if (login.isEmpty() || senha.isEmpty()) {
47            Toast.makeText( context = this,  text = "Informe usuário e senha",  duration = Toast.LENGTH_SHORT).show()
48            return
49        }
50
51        binding.buttonEntrar.isEnabled = false
52        binding.progressBar.visibility = View.VISIBLE
53
54        val client = OkHttpClient()
55        val url = "http://10.0.2.2/Escola2/BackEnd/APIlogin.php"
56
```

ACESSANDO NOSSA API JWT

```
57          val requestBody = FormBody.Builder()
58              .add( name = "login", value = login)
59              .add( name = "senha", value = senha)
60              .build()
61          val request = Request.Builder().url(url).post(requestBody).build()
62
64          Thread {
64              try {
65                  client.newCall(request).execute().use { response ->
66                      val body = response.body?.string().orEmpty()
67
68                      // 401 = credenciais inválidas (a API devolve JSON mesmo no erro)
69                      if (response.code == 401) {
70                          val jsonObj = JSONObject(body)
71                          val erro = jsonObj.optString( name = "erro", fallback = "Usuário ou senha inválidos")
72                          runOnUiThread { mostraErro( msg = erro) }
73                          return@use
74                      }
75
```

ACESSANDO NOSSA API JWT

```
76                    if (!response.isSuccessful)
77                        throw Exception("HTTP ${response.code}")
78
79                    val jsonObj = JSONObject(body)
80                    if (jsonObj.optBoolean( name = "autenticado", fallback = false)) {
81                        val token = jsonObj.getString( name = "token")
82                        val usuario = jsonObj.optString( name = "usuario", fallback = login)
83                        Sessao.salvar( context = this, token, usuario)
84                        runOnUiThread { abrirPrincipal() }
85                    } else {
86                        val erro = jsonObj.optString( name = "erro", fallback = "Usuário ou senha inválidos")
87                        runOnUiThread { mostraErro( msg = erro) }
88                    }
89                }
90            } catch (e: Exception) {
91                runOnUiThread { mostraErro( msg = "Falha na conexão: ${e.message}") }
92            } finally {
93                runOnUiThread {
94                    binding.buttonEntrar.isEnabled = true
95                    binding.progressBar.visibility = View.GONE
96                }
97            }
98        }.start()
99    }
```

ACESSANDO NOSSA API JWT

```
100
101     private fun mostraErro(msg: String) {  3 Usages
102         Toast.makeText( context = this,  text = msg,  duration = Toast.LENGTH_LONG).show()
103     }
104
105     private fun abrirPrincipal() {  2 Usages
106         startActivity(Intent( packageContext = this,  cls = MainActivity::class.java))
107         finish()
108     }
109 }
110
```

ACESSANDO NOSSA API JWT

- Explicando o código:

```
32          // Se já existe token salvo, pula o login
33          if (Sessao.autenticado( context = this))
34              abrirPrincipal()
35      }
```

- Testa se já esta autenticado, se sim, chama a função que abre a tela principal.

```
105     private fun abrirPrincipal() {  2 Usages
106         startActivity(Intent( packageContext = this,  cls = MainActivity::class.java))
107         finish()
108     }
```

- Abre a tela principal (MainActivity)
- Finish() fecha a tela atual (tela de login)

ACESSANDO NOSSA API JWT

```
43          val login = binding.editLogin.text.toString().trim()
44          val senha = binding.editSenha.text.toString()
45
46          if (login.isEmpty() || senha.isEmpty()) {
47              Toast.makeText( context = this,  text = "Informe usuário e senha",  duration = Toast.LENGTH_SHORT).show()
48              return
49          }
```

- Carrega o login e a senha e testa se estão vazios (isEmpty)
- Se pelo menos um deles estiver vazio, exibe uma mensagem e para a execução do método.

ACESSANDO NOSSA API JWT

```
51          binding.buttonEntrar.isEnabled = false
52          binding.progressBar.visibility = View.VISIBLE
```



- Desabilita o botão de entrar enquanto esta tentando validar o usuário e exibe a progressBar.

```
54          val client = OkHttpClient()
55          val url = "http://10.0.2.2/Escola2/BackEnd/APIlogin.php"
56
57          val requestBody = FormBody.Builder()
58              .add( name = "login",  value = login)
59              .add( name = "senha",  value = senha)
60              .build()
61          val request = Request.Builder().url(url).post(requestBody).build()
```

- Prepara os campos para acessar o backend.
- O IP 10.0.2.2 faz referencia ao computador onde o emulador Android esta rodando, para utilização em produção ele deve ser alterado pelo IP real do computador onde esta o backend.

ACESSANDO NOSSA API JWT

```
63      Thread {
64          try {
65              client.newCall(request).execute().use { response ->
66                  val body = response.body?.string().orEmpty()
67
68                  // 401 = credenciais inválidas (a API devolve JSON mesmo no erro)
69                  if (response.code == 401) {
70                      val jsonObj = JSONObject(body)
71                      val erro = jsonObj.optString( name = "erro",  fallback = "Usuário ou senha inválidos")
72                      runOnUiThread { mostraErro( msg = erro) }
73                      return@use
74                  }
```

- Tenta conectar com o backend e guarda a resposta na variável body.
- O .use na linha 65 cria um bloco que será fechado (return@use) em caso de usuário ou senha inválidos.

ACESSANDO NOSSA API JWT

```
76                  if (!response.isSuccessful)
77                      throw Exception("HTTP ${response.code}")
```

- Em caso de erro nos dados recebidos, chama uma exceção passando o código do erro recebido.
- O comando throw força uma exceção

ACESSANDO NOSSA API JWT

```
79              val jsonObj = JSONObject(body)
80              if (jsonObj.optBoolean( name = "autenticado",  fallback = false)) {
81                  val token = jsonObj.getString( name = "token")
82                  val usuario = jsonObj.optString( name = "usuario",  fallback = login)
83                  Sessao.salvar( context = this, token, usuario)
84                  runOnUiThread { abrirPrincipal() }
85              } else {
86                  val erro = jsonObj.optString( name = "erro",  fallback = "Usuário ou senha inválidos")
87                  runOnUiThread { mostraErro( msg = erro) }
88              }
```

- Pega os dados do JSON recebido, salva a sessão (Sessao.salvar) e abre a tela principal (runOnUiThread{ abrirPrincipal()})

ACESSANDO NOSSA API JWT

```
90          } catch (e: Exception) {
91              runOnUiThread { mostraErro( msg = "Falha na conexão: ${e.message}") }
92          } finally {
93              runOnUiThread {
94                  binding.buttonEntrar.isEnabled = true
95                  binding.progressBar.visibility = View.GONE
96              }
97          }
98      .start()
99
```

- Se der algum erro dentro do try, o catch exibe ele.
- Independente de ter erro ou não, o finally abilita novamente o botão e esconde a progressBar

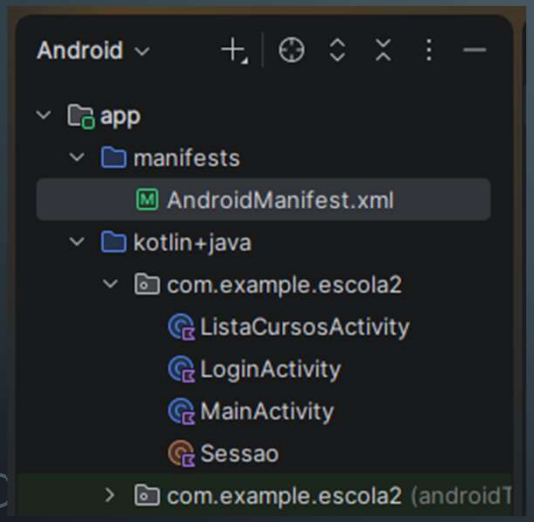
ACESSANDO NOSSA API JWT

- Alterar a tela principal (mainActivity) para checar a sessão:

```
12  class MainActivity : AppCompatActivity(), View.OnClickListener {  3 Usages
13      private lateinit var binding: ActivityMainBinding  3 Usages
14
15      override fun onCreate(savedInstanceState: Bundle?) {
16          super.onCreate(savedInstanceState)
17
18          // Protege a tela: sem token, volta para o login
19          if (!Sessao.autenticado( context = this)) {
20              startActivity(Intent( packageContext = this,  cls = LoginActivity::class.java))
21              finish()
22              return
23          }
24
25          enableEdgeToEdge()
26          binding = ActivityMainBinding.inflate(layoutInflater)
27          setContentView(binding.root)
28          ViewCompat.setOnApplyWindowInsetsListener( view = findViewById(R.id.main)) { v, insets ->
29              val systemBars = insets.getInsets( typeMask = WindowInsetsCompat.Type.systemBars())
30              v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
31              insets
32          }
33          binding.buttonListaCurso.setOnClickListener(this)
34      }
35
```

ACESSANDO NOSSA API JWT

- Alterar o AndroidManifest para abrir primeiro a tela de login:




```
19          <activity
20              android:name=".LoginActivity"
21              android:exported="true">
22              <intent-filter>
23                  <action android:name="android.intent.action.MAIN" />
24
25                  <category android:name="android.intent.category.LAUNCHER" />
26              </intent-filter>
27          </activity>
28          <activity
29              android:name=".ListaCursosActivity"
30              android:exported="false" />
31          <activity
32              android:name=".MainActivity"
33              android:exported="false" />
34      </application>
```

ACESSANDO NOSSA API JWT

- O código da listaCursosActivity tem varias alterações para acessar a backend utilizando o JWT:

```
46      private fun carregaDados() {  1 Usage
47          val token = Sessao.token( context = this)
48          if (token.isNullOrBlank()) {
49              irParaLogin()
50              return
51          }
52
53          val client = OkHttpClient()
54          val url = "http://10.0.2.2/Escola2/BackEnd/APICursosListaTodos.php"
55
56          val requestBody = FormBody.Builder().build()
57          val request = Request.Builder()
58              .url(url)
59              .addHeader( name = "Authorization",  value = "Bearer $token")
60              .post(requestBody)
61              .build()
```

ACESSANDO NOSSA API JWT

```
63          Thread {
64              try {
65                  client.newCall(request).execute().use { response ->
66                      val body = response.body?.string().orEmpty()
67
68                      // 401 = token ausente, inválido ou expirado
69                      if (response.code == 401) {
70                          runOnUiThread {
71                              Sessao.limpar( context = this)
72                              Toast.makeText( context = this,  text = "Sessão expirada. Faça login novamente.",  duration = Toast.LENGTH_LONG).show()
73                              irParaLogin()
74                          }
75                          return@use
76                      }
77
78                      if (!response.isSuccessful)
79                          throw Exception("HTTP ${response.code}")
80
81                      val jsonArray = JSONArray(body)
82                      val listaCursos = ArrayList<String>()
83
```

ACESSANDO NOSSA API JWT

```
                for (i in 0 ≤ ..<jsonArray.length()) {
                    val jsonObj = jsonArray.getJSONObject( index = i)
                    listaCursos.add(jsonObj.getString( name = "nome"))
                }

                runOnUiThread {
                    val adapter = ArrayAdapter( context = this, resource = android.R.layout.simple_list_item_1, objects = listaCursos)
                    binding.listviewCursos.adapter = adapter
                }
            }
        } catch (e: Exception) {
            runOnUiThread {
                Toast.makeText( context = this, text = "Erro ao carregar cursos: ${e.message}", duration = Toast.LENGTH_LONG).show()
            }
        }
    }.start()
}

private fun irParaLogin() {  2 Usages
    startActivity(Intent( packageContext = this, cls = LoginActivity::class.java))
    finish()
}
```